

An Efficient Low-Latency Algorithm and Implementation for Rate-Matching and Bit-Interleaving in 5G NR

Khitish C Behera
Member, IEEE
khitish.behera@ieee.org

Abstract—5G New Radio (NR) is primarily characterized by multi-Gbps throughput and up to 10X lower latency than LTE. 5G adopts Low-Density Parity Check (LDPC) code as the channel coding candidate for data channels. Rate-Matching and Bit-Interleaving functions are performed after LDPC encoding in the transmit signal chain. The purpose of Rate Matching is to select a specific set of encoded bits for transmission by the process of puncturing and/or repetition to support HARQ operation. The output of the Rate-Matching buffer goes through row-column permutation function in Bit-Interleaving process. The rate-matched bits from the circular buffer are written in row-first order into another buffer and read in column-first order. While copying the bits from the rate-matching circular buffer, the filler bits are skipped and does not enter the row-column buffer. The rate-matching buffer to row-column buffer copy operation accounts for the overall latency in the transmit chain. This paper addresses the latency aspects while processing the large Transport Blocks corresponding to the maximum downlink (DL) throughput. An efficient M-parallel look-ahead pointers generation algorithm is proposed to read M-interleaved bits directly from rate-matching buffer, avoiding row-column permutation operation, and thus the need of a separate buffer, where M is programmed for a target latency.

Index Terms—5G, bit-interleaving, circular-buffer, latency, rate-matching

I. INTRODUCTION

While the fourth generation (4G) long term evolution (LTE) focused on mobile broadband, the fifth-generation cellular technology known as 5G NR aims at expanding the use cases of mobile communications. 5G NR supports three families of use cases: enhanced mobile broadband (eMBB), massive machine type communications (mMTC) and Ultra-Reliable Low Latency Communications (URLLC). The key performance requirements to be able to address the traffic for these use case scenarios are defined in 3GPP specification [2]. The peak data rate requirement is 20 Gbps for downlink (DL) and 10 Gbps for uplink (UL) [2]. The user plane latency requirement is 4 ms for eMBB and 0.5 ms for URLLC. At the physical layer, a frame structure with a scalable slot duration as listed in Table I are defined to address diverse latency and Quality of Service (QoS) requirements. Taking latency and throughput requirements into considerations, NR has adopted LDPC codes as the channel coding candidate for data channels [4] [8] [9] [10]. In the DL

TABLE I
NR SCALABLE SLOT DURATION

Numerology	sub-carrier-spacing (SCS)	# slots per sub-frame	slot duration
0	15 kHz	1	1 ms
1	30 kHz	2	0.5 ms
2	60 kHz	4	0.25 ms
3	120 kHz	8	0.125 ms

encoder processing chain, as shown in Fig. 1 rate matching is applied separately on each code block. Like LTE, circular buffer-based rate-matching to support Incremental Redundancy Hybrid Automatic Repeat Request (IR-HARQ) is defined for NR LDPC codes. The bit selection algorithm is based on the redundancy versions (RV) corresponding to different starting positions in the transmit circular buffer. While in LTE sub-block interleaving is applied before rate-matching function, bit-interleaving function in NR is performed after bit selection process completes. The interleaved bits of code blocks are concatenated before performing scrambling and modulation functions in the processing chain. Rate-matching algorithms based on circular buffer for LTE have been analyzed well in [7]. There is prior work in [5] that analyzes the circular buffer rate matching and bit selection algorithms for 5G LDPC codes. As NR defines a scalable slot-based scheduling, with slot duration scaling with SCS, e.g. slot duration is 1 ms for 15 kHz SCS and 0.5 ms for 15 kHz numerology, the latency requirements in 5G NR DL processing chain are stringent. Rate-matching and bit-interleaving are the latency sensitive operations in DL transmit processing chain. The prior work in [6] proposes an efficient rate-matching implementation for LTE. The prior work in [11] analyzes decoding latency of LDPC codes, work in [15] proposes a decoding schedule for HARQ application to improve decoder throughput. The previous literatures in [5] [13] have investigated the HARQ performance of re-transmissions and the retransmission size of soft HARQ. The latency attributed by underlying rate-matching and bit-interleaving algorithms in DL encoding processing chain for 5G NR has not been investigated so far. This article analyzes the latency aspects of the circular buffer rate-matching and bit-interleaving algorithms for the DL encoder processing chain.

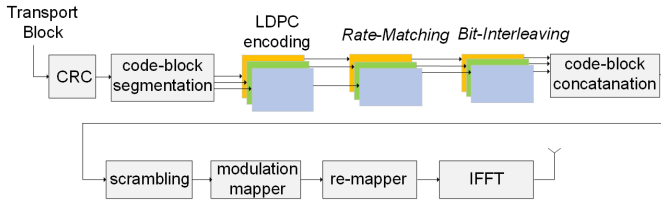


Fig. 1. Downlink processing chain for data payload.

Section II summarizes NR LDPC, studies rate-matching, and bit-interleaving algorithms, and investigates the underlying latency issues while processing a transport block in the DL transmit processing chain. Section III formulates a novel algorithm for an efficient implementation to be able to scale the latency by a programmable factor. Section IV demonstrates FPGA implementation of the proposed parallel algorithm. Finally, conclusion remarks are presented in Section V.

II. PROBLEM FORMULATION

A. NR LDPC

NR defines raptor-like rate-compatible LDPC codes [9]. Two base graphs, BG1 and BG2 are defined to handle wide range of payloads sizes and code rates efficiently. The maximum code block size is 8448 bits for BG1 and 3840 bits for BG2. BG1 has dimension 46×68 with mother code rate of $1/3$. BG2 has dimension 42×52 with mother code rate of $1/5$. The first 22 columns of BG1 and 10 columns of BG2 correspond to information bits. The first two columns of both base graphs are not transmitted. The general structure as shown in Fig. 2 is same for both base graphs. Sub matrix A correspond to systematic bits. D has a weight three in the first column, the remaining columns have a dual-diagonal structure. B corresponds to single parity check based extension that is used to support lower rates for IR-HARQ. There are 51 lifting sizes defined to handle wide a range of payload sizes [1]. For a given lifting size Z , each “1” in the base graph is replaced by $Z \times Z$ identity matrix circularly shifted by the corresponding shift coefficient and each “0” in the base graph is replaced by $Z \times Z$ all-zero matrix. For payload sizes that are outside the range of 51 native defined lifting sizes, filler bits (zero-padding) are appended to the code block before encoding. The filler bits are skipped in the rate-matching procedure.

B. DL Rate-Matching in 5G NR

The encoded bits of the code block are written into a circular buffer of length N_{cb} . To reduce the complexity of the buffer, a limited buffer rate matching (LBRM) is proposed in NR. If LBRM is disabled, $N_{cb} = N_{ldpc}$, where N_{ldpc} is $66Z$ for BG1 and $50Z$ for BG2. If LBRM is enabled, N_{cb} is equal to

$$N_{cb} = \min(N_{ldpc}, \lfloor \frac{TBS_{LBRM}}{CR_{LBRM}} \rfloor) \quad (1)$$

where $R_{LBRM} = 2/3$ and TBS_{LBRM} is a function of number of layers, modulation order and the number of physical resource blocks. TBS_{LBRM} is determined in higher layers according to

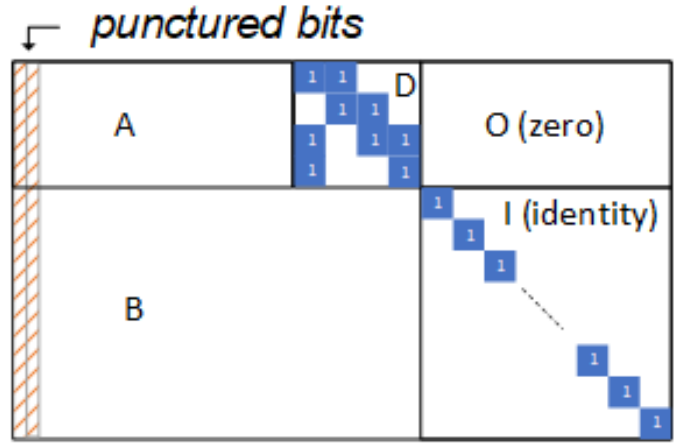


Fig. 2. NR LDPC base graph structure.

subclause 5.1.3.2 in [3]. A technique is employed where the modulation coding scheme (MCS), which achieves the highest data rate (corresponding to maximum transport block size) while maintaining a target block error rate, is selected. The determination of rate matching sequence length E_r is done as given in sub clause 5.4.2.1 in [1]. To support IR-HARQ through 4-RVs (0,1,2,3), fixed locations in the circular buffer are defined at $\{0,17,33,56\} \times Z$ for base graph 1 and at $\{0,13,25,43\} \times Z$ for base graph 2. During the selection process the filler-bits in the encoded bit stream are skipped.

C. DL Channel Bit-Interleaving

The rate-matching bits of each code block from the circular buffer are reordered by a row-column interleaver. The number of rows in the interleaver is given by the modulation order Q_m (2/4/6/8) and thus the bits in one column correspond to one modulation symbol. The bits from the rate-matching circular buffer are written row-by-row into this row-column buffer and read out column-by-column. The reading process can start after all the rows of the buffer are filled.

D. Processing cost of rate-matching and bit-interleaving procedures

The underlying operations of rate-matching and bit-interleaving algorithms are explained in Fig. 3. The bit selection process removes the filler-bits before reading from the rate-matching circular buffer. The process of skipping or removing filler-bits will need additional processing, incurring delay in the reading process. The bits read from the buffer are written into the row-column buffer in the next step. But reading from the second buffer cannot start until all the rows of the buffer are filled in. This introduces significant delay. The magnitude of delay is directly proportional to the total size of the rate matching bits E_r . Apart from the delay incurred, bit-interleaving process needs a separate buffer of size E_r bits. As NR also specifies removing the first $22Z$ bits information bits, where Z can be maximum 384 bits, thus removing up-to 768 bits from the beginning of the encoder stream, costs additional cycles due to shift-operations.

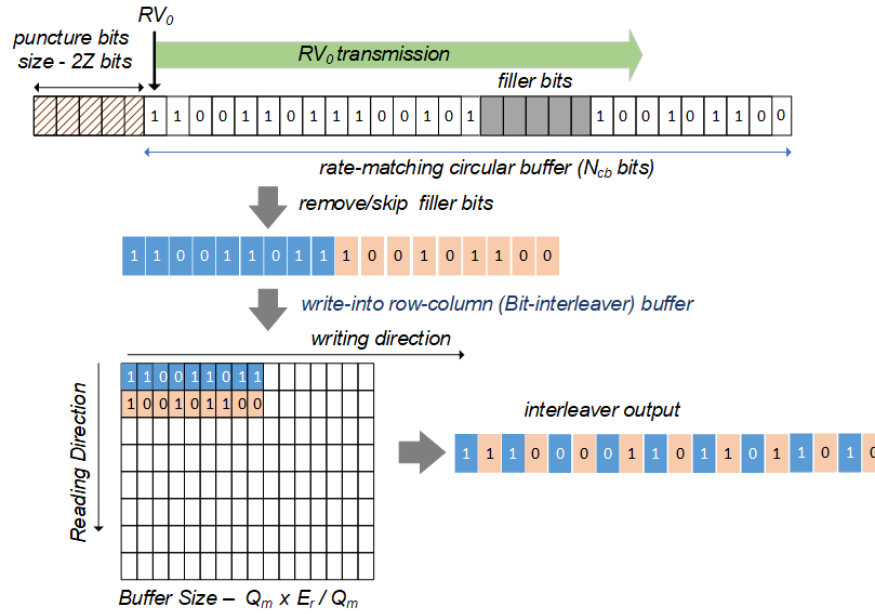


Fig. 3. Rate-matching and bit-Interleaving operations in NR: $RV_{id} = 0$, $E_r = 18$, and $Q_m = 2$ (QPSK)

As both rate-matching and bit-interleaving operate on a CB basis, it is important to study the accumulated delay effect while processing several CBs in sequence for a transport block corresponding to highest throughput. This is examined for a specific transport block size for 0.5 ms slot. The size of a Transport Block (TB) based on Table 1 in [6] for 4-layer, 273RB allocation and 256-QAM is 1277992 bits. The total number of CBs for this TB that need to be processed in DL transmit chain in a 0.5 ms slot is 152, where each CB size is 8448 bits. The re-ordering process in a typical implementation is done bit-by-bit, by reading from circular-buffer and writing into another row-column buffer. Considering total number of bits to be transmitted is 16000, a typical implementation may cost 16000 cycles for writing into the row-column buffer, in a hardware with limited clock frequency such as a FPGA running at a clock of about 400 MHz, it takes $16000 \times 2.5 \text{ ns} = 40 \text{ us}$ per CB. Thus, for 12 CBs just the write operations will take 0.48 ms, leaving very little margin for other modules in DL transmit chain. Additional processing of removing puncture bits and filler-bits would further incur delay.

The hardware implementations using ASIC solutions or FPGA prototypes address the latency issue in the DL processing chain by scaling the hardware resources. For example, in an ASIC chip, which benefits from technology node miniaturization, can operate at a clock in the range of GHz frequencies, cuts-down the latency straightaway by a factor of about 2.5 as opposed to a hardware operating at 400 MHz clock frequency. Higher operating clock frequency is complemented by parallel processing. For example, an ASIC chip can schedule 19-CBs across 8-parallel engines, thus scaling total latency by 8, but increasing the hardware resource by 8-times. By running the hardware at GHz clock frequency to scale the latency is not an efficient solution. In a FPGA solution, which comes with

limited logic resources, and with a restricted and fixed clock frequency in the range of 400-600 MHz, meeting the latency requirement is a big challenge.

III. PROPOSED ALGORITHM : M-PARALLEL POINTER GENERATION FOR RATE-MATCHING BUFFER

A. An efficient low-latency algorithm for rate-matching and bit-interleaving

To address the latency issues, a parallel algorithm for rate-matching and bit-interleaving is proposed. Buffer-to-buffer copy operation which is latency sensitive is avoided, instead a set of M-parallel pointers are generated on the fly, where M is a programmable value based on output latency requirements across different modulation orders and operating clock frequencies. As an example, considering M bits are read in parallel, for reading rate-matching sequence of size 16000 bits from the buffer, with M set to 8, the total read time with a clock frequency of 400 MHz (period=2.5ns) will be $(16000/8) \times 2.5 \text{ ns} = 5 \text{ us}$. Accordingly, the read latency will scale to 2.5 ns if 16-parallel pointers are used for reading. By look-ahead computation, interleaved addresses can be generated that are used to read circular buffer in parallel. As an example, considering modulation order (Q_m) as 256-QAM, the 8-parallel interleaved offset addresses are generated at addresses E_r/Q_m , $2 \times E_r/Q_m$, $3 \times E_r/Q_m$, $4 \times E_r/Q_m$, $5 \times E_r/Q_m$, $6 \times E_r/Q_m$ and $7 \times E_r/Q_m$ starting at RV_{id} index. The proposed algorithm applies 3-steps in sequence: *pointer initialization*, *on-the-fly increment*, and *bit-rearrangement*. The algorithms are generalized assuming LDPC encoder does not puncture 2Z bits. It is to be noted that E_r , E_r/Q_m , N_{cb} , RV_{id} , n_{Filler_st} and n_{Filler_end} and N_{cb} are known parameters, where E_r is the total number of rate-matching bits, Q_m is the modulation order (1/2/4/6/8 based on BPSK/QPSK/16-QAM/64-QAM/256-QAM), RV_{id} is starting

address in the circular buffer, $n\text{Filler_st}$ and $n\text{Filler_end}$ are indices of the filler bits region, and N_{cb} is the number of bits in encoded block.

1) *M-parallel pointer initialization*: In a system where LDPC encoding function does not puncture the built-in information bits or remove the filler bits, these puncture bits of size $2Z$, are stored in the rate matching buffer. At the start of reading process, the higher layer parameters such as RV_{id} , $n\text{Filler_st}$, $n\text{Filler_end}$ pointers and the code-block size parameter N_{cb} are pre-incremented by $2Z$. Figures 4 - 8 illustrate the pointer initialization process for modulations: BPSK, QPSK, 16-QAM, 64-QAM and 256-QAM. To illustrate the concept, the output data width (M) is selected as 8. The value of M is chosen such that it is a least common multiple of Q_m values (1/2/4/6/8) and divides E_r . In the case when M is chosen as a multiple of the highest Q_m value, some of Q_m values may not divide M , e.g. when M is 8, $Q_m(=6)$ (for 64-QAM modulation) does not divide 8, thus only 6 out of 8 pointers are used in 64-QAM mode, whereas in BPSK/QPSK/16-QAM modulations, all of 8-pointers are used. The idea is that the buffer will output M parallel bits corresponding to M pointers, and M bits should constitute an integer number of symbols based on the modulation order. For example, if $M=24$, 24 pointers will be generated, equivalently 24 bits read from the buffer will constitute 24 symbols in BPSK modulation, 12 symbols in QPSK modulation, 6 symbols in 16-QAM, 4 symbols in 64-QAM and 3 symbols in 256-QAM modulation modes.

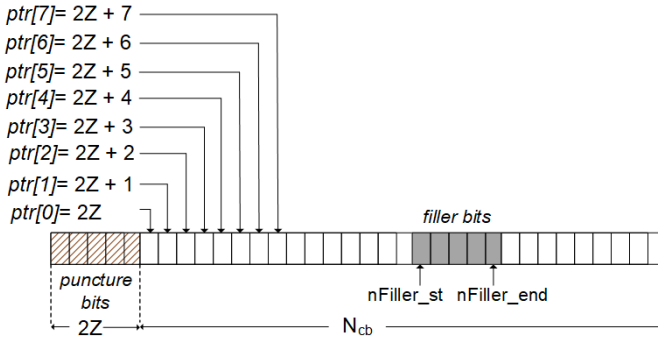


Fig. 4. $RV_{id}=0$, $Q_m=1$ (BPSK), 1-set of 8-parallel pointers initialized.

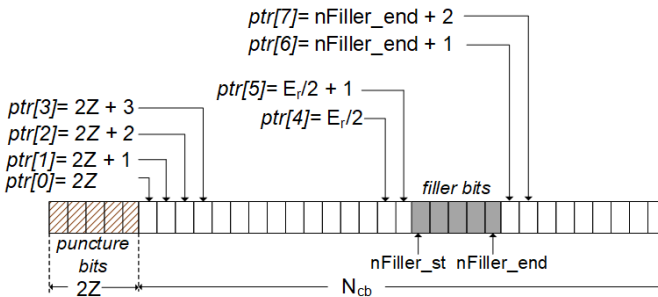


Fig. 5. $RV_{id}=0$, $Q_m=2$ (QPSK), 2-set of pointers with 4-sequential pointers in each set are initialized.

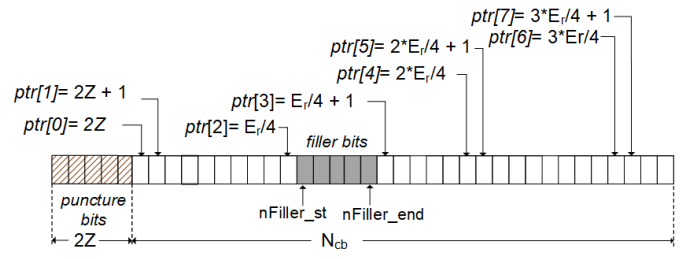


Fig. 6. $RV_{id}=0$, $Q_m=4$ (16-QAM), 4-set of pointers with 2-sequential pointers in each set are initialized.

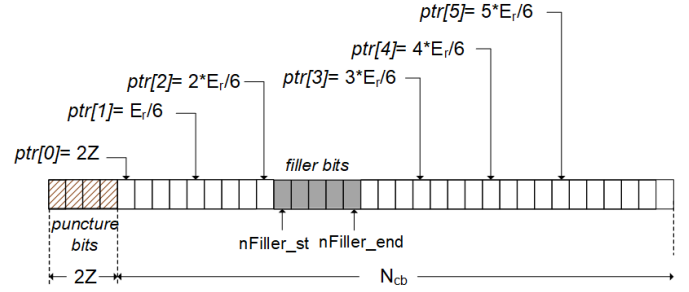


Fig. 7. $RV_{id}=0$, $Q_m=6$ (64-QAM), 6-parallel pointers are initialized.

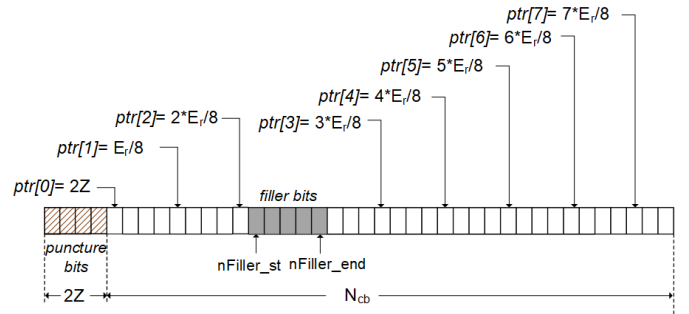


Fig. 8. $RV_{id}=0$, $Q_m=8$ (256-QAM), 8-parallel pointers are initialized.

Algorithm 1 is a generalized M -pointer initialization procedure. As can be seen in Fig. 4, in BPSK modulation, 8-sequential pointers are initialized starting with address pointed by RV_{id} . In the case of QPSK, 2-sets, with 4-pointers in each set are initialized starting from 2-interleaved offset addresses at 0 and E_r/Q_m . The pre-computed pointers are checked for wrap-around condition, and if occurs, the pointers are incremented such as to skip $2Z$ bits. In case the pointers fall in the filler region, by look-ahead computing the offsets from filler window end index pointers are updated. Extending the algorithm to 16-QAM modulation, where 4-sets of pointers with 2-pointers in each set are initialized starting from interleaved offset addresses at 0, E_r/Q_m , $2 \cdot E_r/Q_m$ and $3 \cdot E_r/Q_m$. In the case of 64-QAM, 6 out-of-8, parallel pointers are initialized starting at addresses 0, E_r/Q_m , $2 \cdot E_r/Q_m$, $3 \cdot E_r/Q_m$, $4 \cdot E_r/Q_m$ and $5 \cdot E_r/Q_m$ offset from RV_{id} . Likewise, the look-ahead scheme is extended for 256-QAM modulation as explained in Fig. 8.

Algorithm 1 M-parallel pointer Initialization

Require: $M = \text{Number of parallel pointers}$

```
Qm ← 1/2/4/6/8
N ← (M/Qm)
RVid ← RVid + 2Z
Ncb_int ← Ncb + 2Z
nFillerst ← nFillerst + 2Z
nFillerend ← nFillerend + 2Z
for  $i = 0 \text{ to } Q_m - 1$  do
  for  $j = 0 \text{ to } N - 1$  do
    incr ← 0
    for  $k = 0 \text{ to } j$  do
      if  $(n\text{Filler}_{st} \leq (RV_{id} + i * (E_r/Q_m) + k)) \leq n\text{Filler}_{end}$  then
        incr ++
      end if
    end for
    if  $(RV_{id} + i * (E_r/Q_m) + j) > N_{cb\_int}$  then
      ptr[ $i * N + j$ ] ← RVid +  $i * (E_r/Q_m) + j - N_{cb}$ 
    else if  $(incr \neq 0)$  then
      ptr[ $i * N + j$ ] ← nFillerend + incr
    else
      ptr[ $i * N + j$ ] ← RVid +  $i * (E_r/Q_m) + j$ 
    end if
  end for
end for
```

Algorithm 2 M-parallel pointer Increment

Require: $M = \text{Number of parallel pointers}$

```
Qm ← 1/2/4/6/8
N ← (M/Qm)
Ncb_int ← Ncb + 2Z
nFillerst ← nFillerst + 2Z
nFillerend ← nFillerend + 2Z
for  $i = 0 \text{ to } Q_m - 1$  do
  for  $j = 1 \text{ to } N$  do
    incr ← 0
    for  $k = 1 \text{ to } j$  do
      if  $(n\text{Filler}_{st} \leq (ptr[i * N + N - 1] + k) \leq n\text{Filler}_{end})$  then
        incr ++
      end if
    end for
    if  $((ptr[i * N + N - 1] + j) > N_{cb\_int})$  then
      ptr[ $i * N + j - 1$ ] ← ptr[ $i * N + N - 1$ ] +  $j - N_{cb}$ 
    else if  $(incr \neq 0)$  then
      ptr[ $i * N + j - 1$ ] ← nFillerend + incr
    else
      ptr[ $i * N + j - 1$ ] ← ptr[ $i * N + N - 1$ ] +  $j$ 
    end if
  end for
end for
```

2) *M-parallel pointer increment:* M-parallel pointers are split into Q_m sets and with each set having M/Q_m sequential

Algorithm 3 M-parallel bits decode and re-arrange

Require: $M = \text{Number of parallel pointers}$ $W = \text{rm_buffer word - width /*in-power-of-2}$

e.g. 16/32/64 */

 $Q_m \leftarrow 1/2/4/6/8$ $N \leftarrow (M/Q_m)$ Case (Q_m)

1 : /*BPSK */

bit[0] ← rm_buffer[ptr[0]%W]

bit[1] ← rm_buffer[ptr[1]%W]

...

bit[M-1] ← rm_buffer[ptr[M-1]%W]

2 : /*QPSK */

bit[0] ← rm_buffer[ptr[0]%W]

bit[1] ← rm_buffer[ptr[N]%W]

bit[2] ← rm_buffer[ptr[1]%W]

bit[3] ← rm_buffer[ptr[1 + 1 * N]%W]

...

bit[M-1] ← rm_buffer[ptr[M-1]%W]

4 : /*16-QAM */

bit[0] ← rm_buffer[ptr[0]%W]

bit[1] ← rm_buffer[ptr[N]%W]

bit[2] ← rm_buffer[ptr[2 * N]%W]

bit[3] ← rm_buffer[ptr[3 * N]%W]

bit[4] ← rm_buffer[ptr[1]%W]

bit[5] ← rm_buffer[ptr[1 + 1 * N]%W]

bit[6] ← rm_buffer[ptr[2 + 2 * N]%W]

...

bit[M-1] ← rm_buffer[ptr[M-1]%W]

6 : /*64-QAM */

bit[0] ← rm_buffer[ptr[0]%W]

bit[1] ← rm_buffer[ptr[N]%W]

bit[2] ← rm_buffer[ptr[2 * N]%W]

...

bit[5] ← rm_buffer[ptr[5 * N]%W]

bit[6] ← rm_buffer[ptr[1]%W]

bit[7] ← rm_buffer[ptr[1 + 1 * N]%W]

...

bit[M-1] ← rm_buffer[ptr[M-1]%W]

8 : /*256-QAM */

bit[0] ← rm_buffer[ptr[0]%W]

bit[1] ← rm_buffer[ptr[N]%W]

bit[2] ← rm_buffer[ptr[2 * N]%W]

...

bit[7] ← rm_buffer[ptr[7 * N]%W]

bit[8] ← rm_buffer[ptr[1]%W]

bit[9] ← rm_buffer[ptr[1 + 1 * N]%W]

...

bit[M-1] ← rm_buffer[ptr[M-1]%W]

pointers. After initialization phase, sequential pointers in each set are incremented *on-the-fly*. The sequential pointers in each set corresponding to modulation order Q_m are checked to find if it falls in the filler region or wraps around to the built-in puncture region. The increment value for sequential pointers to skip filler region is found by look-ahead computation.

3) *M-bit parallel decode*: In the third phase, M -words from the rate matching buffer are read in parallel. It can be safely assumed that memory word width (W) value is a power of two. Algorithm 3 is a generalized procedure to decode the bit position using modulus operation. Since modulus operation using power of two in hardware reduces to a simple operation, the decoding function can be done as soon as data is read from memory. The re-arrange operation is a straightforward regrouping operation based on the modulation order as explained in the Algorithm 3.

IV. HARDWARE IMPLEMENTATION OF THE ALGORITHM

A. FPGA implementation

The overall architecture of the proposed algorithm is shown in Fig 9. Encoder design output rate is 64-bits every clock cycle, thus the rate-matching memory is designed with word width of 64-bits. LDPC encoder design does not puncture $2Z$ bits and are stored in the rate-matching buffer. As the encoder outputs at the base code rate of $1/3$ for BG1, considering the max CB size of 8448 bits and max Z as 384 bits, the rate-matching buffer of size $26112(8448*3+2*384)$ bits is designed with depth of 408 words, with each word of 64 bits. With a parallelism of 8 i.e. $M=8$, pointer generation logic generates 8-parallel pointers for the rate-matching buffer. The 3-operations as described in the proposed algorithms such as initialization, *on-the-fly* parallel increment and decode-and-rearrange are done in 3-separate clock cycles, initialization is done one time at the start CB processing, whereas the increment and bit decoding, re-arrange functions are pipe-lined in the implementation.

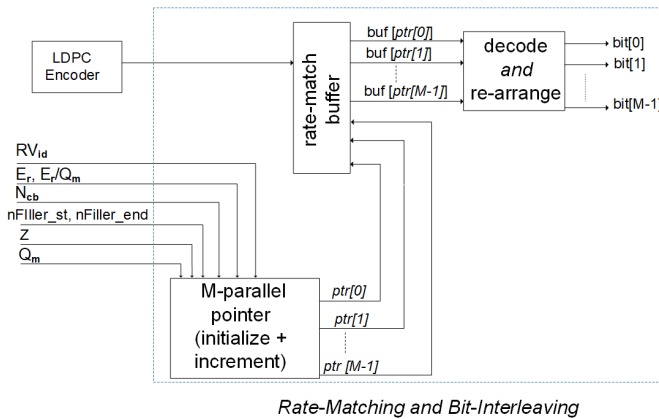


Fig. 9. Overall architecture for the proposed parallel algorithm.

As shown in Fig 8, in 256-QAM modulation, 8-parallel pointers are generated and as can be shown, that since 2nd pointer (ptr[1]) matches with filler region starting index, it is

incremented to one location next to filler region end index i.e. nFiller_end+1.

On the memory read path, modulus operation is applied on the pointer values to decode the bit from 64-bit word. Since the number of output parallel word width ($M=8$) is same as the modulation order ($Q_m=8$) in 256-QAM modulation case, the bit arrangement function directly drives the 8-bit outputs. This is illustrated on Fig 11.

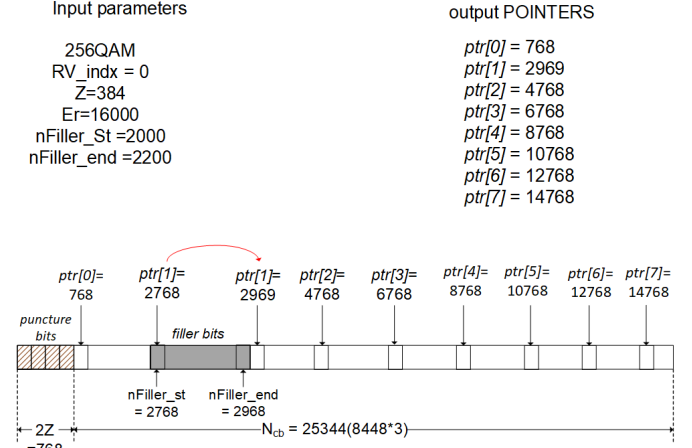


Fig. 10. A working example for 8-parallel pointer generation and update.

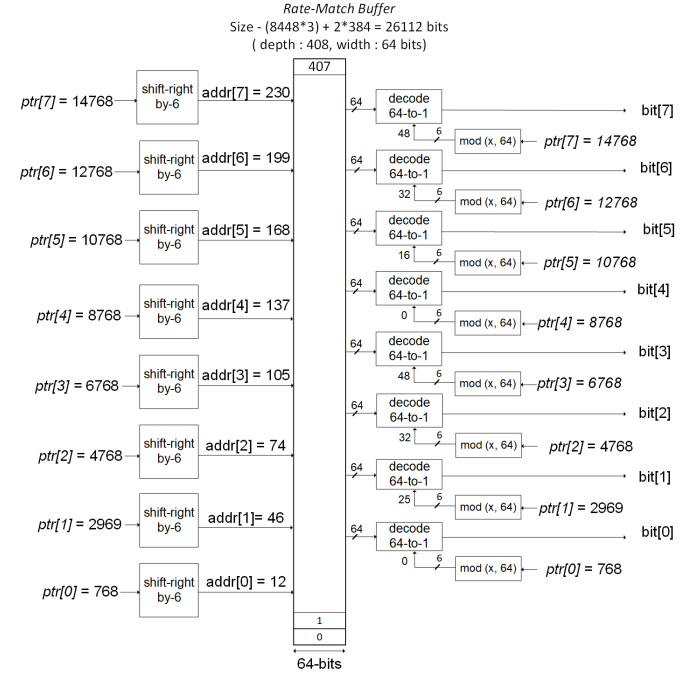


Fig. 11. 8-parallel bit decode architecture is illustrated for 256-QAM modulation.

B. Resource Utilization

The proposed algorithm is implemented in RTL using Verilog on Xilinx Zynq UltraScale ZCU106(zcu7ev-ffvc1156-2-e) evaluation board. Table II gives a summary of resource

utilization. The total latency to process 16000 rate-matching bits is 5 microseconds for 256-QAM modulation, which is 2000 cycles at 400 MHz clock. The processing time is reduced by 1/8, due to 8-way parallel pointer processing. In 64-QAM modulation mode, the hardware uses 6 out of 8 parallel pointers, thus processing time will be reduced by 1/6.

C. Hardware Limitations

The algorithm proposed in this paper is generalized for parallelism factor M, it allows to address the scenario when LDPC encoder design does not remove the puncture bits and filler bits. The higher M values scale the processing time, but also increases the computation logic required for look-ahead computation to find the increment value and *on-the-fly* updating the pointers. This presents a challenge to meet timing in hardware. For example, the same design on FPGA cannot meet timing at 400 MHz for 24-way (M=24) parallel pointers. If the encoder hardware removes the puncture bits and filler bits from the encoding stream, the computations will be simplified and look-ahead logic to find the offset from filler region will not be required, thus it will enable using a higher M value. The M-way parallel access requires M words to be read in parallel from rate-matching memory. If the physical memory cell cannot support multiple read ports, redundant memory instances are required.

TABLE II
RESOURCE UTILIZATION

Resource	Utilization	Available	Utilization %
LUT	7364	230400	3.2
FF	614	460800	0.13
BRAM	16	312	5.13
LUTRAM	56	101760	0.06

V. CONCLUSION

This paper addresses the latency sensitive operations in rate-matching and bit-interleaving functions in 5G NR. The buffer-to-buffer copy operations that are typically employed for the row-column permutations are latency critical. By look-ahead computing M-parallel pointers, the latency for each code-block is reduced by a factor of 1/M. Also, by avoiding a separate buffer for row-column permutations, the proposed algorithm leads to an efficient implementation. This generalized algorithm gives a flexibility to scale the latency while processing large Transport Blocks in the DL encoding chain. The algorithm is demonstrated on Xilinx Zynq UltraScale ZCU106 evaluation board running at a clock frequency of 400 Mhz. For a typical case of E_r of size 16000 bits, the total latency is reduced to 2000 cycles from 16000 cycles by employing 8-way parallel look-ahead pointer computation.

REFERENCES

- [1] "NR; Multiplexing and channel coding (Release 15)," TS 38.212, V15.4.0, 3GPP, Dec. 2018.
- [2] "5G; Study on scenarios and requirement for next generation access technologies (Release 15)," TS 38.913, V15.0.0, 3GPP, 2018.
- [3] "NR; Physical layer procedures for data(Release 15)," TS 38.214, V15.4.0, 3GPP, Dec. 2018.
- [4] Dennis Hui, Sara Sandberg, Yufei Blankenship, Mattias Andersson, Leefke Grosjean, "Channel coding in 5G new radio: a tutorial overview and performance comparison with 4G LTE," *IEEE Vehicular Technology Magazine*, Volume 13, Issue 4, Dec. 2018.
- [5] Fatemeh Hamidi-Sepehr, Ajit Nimbalkar, Gregory Ermolaev, "Analysis of 5G LDPC Codes Rate-matching Design," in *Proc. IEEE 87th Vehicular Technology Conference (VTC Spring)*, Porto, Portugal, June 2018.
- [6] Chixiang Ma, Ping Lin, "Efficient Implementation of Rate Matching for LTE Turbo Codes," *2nd International Conference on Future Computer and Communication*, 21-24 May 2010.
- [7] Cheng, J. F., Nimbalkar, A., Blankenship, Y., Classon, B., Blankenship, T. Keith, "Analysis of Circular Buffer Rate Matching for LTE Turbo Code," *68th Vehicular Technology Conference*, Calgary, Canada, 21-24 Sept. 2008.
- [8] Tom Richardson and Shrinivas Kudekar, "Design of low-density parity check codes for 5G new radio," in *IEEE Communication Magazine*, vol. 56, no. 3, pp. 28-34, March 2018.
- [9] Sung Ik Park, Yiyang Wu, Heung Mook Kim, Namho Hur, Jeongchang Kim, "Raptor-Like Rate Compatible LDPC Codes and Their Puncturing Performance for the Cloud Transmission System," *IEEE Transactions on Broadcasting*, vol. 60, no. 2, pp. 239-245, June 2014.
- [10] Hao Wu, Huayong Wang, "A High Throughput Implementation of QC-LDPC Codes for 5G NR," in *IEEE Access* (vol. 7), Dec. 2019.
- [11] Hao Wu, Huayong Wang, "Decoding Latency of LDPC Codes in 5G NR," *Proc. 29th International Telecommunication Networks and Applications Conference (ITNAC)* Auckland, New Zealand, 27-29 Nov. 2019.
- [12] Tram Thi Bao Nguyen and Tuy Nguyen Tan and Hanho Lee, "Efficient QC-LDPC encoder for 5G new radio," *Electronics* vol. 8, no. 6, pp. 668, June 2019.
- [13] Chulong Liang, Jin Xu, Qiujin Guo, Jun Xu, "Soft HARQ for 5G Downlink," *International Wireless Communications and Mobile Computing (IWCMC)*, Limassol, Cyprus, 15-19 June 2020.
- [14] Qianfan Wang, Suihua Cai, Li Chen, Xiao Ma, "A Throughput-Enhanced HARQ Scheme for 5G System via Partial Superposition," *IEEE Communications Letters*, 23 June 2020.
- [15] Cing-Yi Liang, Mao-Ruei Li, Huang-Chang Lee, Hsin-Yu Lee, Yeong-Luh Ueng, "Hardware-friendly LDPC Decoding Scheduling for 5G HARQ Applications," *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* Brighton, United Kingdom, 12-17 May 2019.